USER MANUAL

SPARKINJECT

SparkInject is a perpetual-license Sparkplug-to-cloud injection module, landing estate data in Snowflake or Databricks with store-and-forward and exactly-once semantics.

PERPETUAL LICENSE IGNITION 8.1.19+ AND 8.3 RUNS ON: HOST

CONTENTS

1. OVERVIEW

SparkInject is a perpetual-license Sparkplug-to-cloud injection module for Ignition. It consumes `spBv1.0/#` as an ordinary MQTT client, decodes it with the same codec the SparkBridge Host uses, and lands it in Snowflake or Databricks with store-and-forward across a cloud outage and exactly-once semantics across a restart. The default sink writes local JSONL files so the pipeline can be proved end to end before a warehouse is provisioned.

The MQTT ingest thread only decodes and does non-blocking offers; a stalled cloud destination can never back-pressure the local MQTT dispatch loop. A bounded queue and disk buffer absorb the outage instead.

- A SparkBridge Host gateway.
- A Snowflake or Databricks destination, provisioned before switching off the default JSONL sink.
- For Snowflake: a role and a key-pair user with `USAGE` and `INSERT, SELECT` grants, created from the module's generated DDL.

3. INSTALL

1. SparkInject carries its own license. Install the signed module through Config > Modules > Install or Upgrade a Module; no build step is required.
2. Set `sparkinject.enabled=true`; it is off by default.
3. Leave `sparkinject.sink=jsonl` (the default) to prove the pipeline end to end before provisioning a warehouse.

Perpetual license. See the README in the bundle for purchasing. Standard suite key behavior applies: the file lives at `<data>/sparkbridge-license.key` and is re-checked every 30 seconds.

5. CONFIGURE

Before switching `sparkinject.sink=snowpipe`, run the generated DDL at `docs/sparkinject-schema.sql` to create the database, schema, and one table per mapped group, plus the role and key-pair user. The module cannot create tables itself.

SPARKINJECT.ENABLED	false by default.
SPARKINJECT.SINK	`jsonl` (default) or `snowpipe`; `databricks` for that destination.

SPARKINJECT.MAPPING.FILE	default `data/sparkinject-mapping.properties`, group-to-table routing.
SPARKINJECT.MAPPING.REQUIRERULE	false by default; true refuses an unmapped group instead of deriving a table name.
SPARKINJECT.SECRETS.FILE	default `data/sparkinject-secrets.properties`, mode 600, never on the JVM command line.
SPARKINJECT.BROKER.URI	default `tcp://127.0.0.1:1883`.
SPARKINJECT.SNOWFLAKE.PRIVATEKEY	secrets-file key; PEM PKCS#8 private key for the Snowflake key-pair user.
SPARKINJECT.STATUS.TOKEN	secrets-file key; bearer token for status.json, unset means loopback-only.

The landed table shape is one narrow EAV table per Sparkplug group, with columns for node key, device, metric, source timestamp, numeric/string/boolean value, quality (192 good, or a stale/death-marker code) and the Sparkplug `is_historical` flag.

SparkInject ingests passively (no STATE, no Will) and batches into the configured sink through a bounded queue. A cloud outage spills to disk via the store-and-forward buffer and replays the missed suffix on reconnect, using an offset-token discipline so Snowflake is told exactly what it has not received rather than replaying everything on disk.

7. VERIFY

Check `GET /data/io.sparkinject.snowflake/status.json`. A lane in state `BLOCKED` always carries a non-null `problem` field with pasteable remediation, for example a missing destination table. `inject.buffered` reports bytes currently held on disk and is the number to alarm on.

SYMPTOM	CAUSE	FIX
	`sparkinject.enabled` is false (default)	Set it to true
	Destination table or schema does not exist yet	Read `status.json`'s `prc` field and run the missing from `docs/sparkinject` `schema.sql`
	Cloud destination unreachable or credentials invalid	Check Snowflake/Databri connectivity and the key i `sparkinject.secrets.`

SYMPTOM	CAUSE	FIX
	`sparkinject.mapping.requireRule` is false (default)	Set it to true to refuse unmapped groups instead add the mapping

9. REFERENCE

- Status route: `GET /data/io.sparkinject.snowflake/status.json`.
- Secrets file default: `data/sparkinject-secrets.properties` (must be mode 600).
- Mapping file default: `data/sparkinject-mapping.properties`.
- Generated DDL: `docs/sparkinject-schema.sql` in the module bundle.